

유동연

프론트엔드 개발자 · SAFEAI

dong-coding.vercel.app · github.com/easyDong19



개발자 유동연입니다.

좋은 개발은 단순히 코드를 잘 작성하는 것을 넘어,
시간과 비용, 팀의 상황, 유지보수 가능성, 사용자 경험 사이에서
적절한 균형을 찾는 일이라고 생각합니다.

경력

SAFEAI 프론트엔드 개발자
2024.07 – 2026.10
산업·공공 안전 AI 솔루션

학력

한양대학교 산업공학과 학사
2019 – 2025
한국디지털미디어고등학교 해킹방어과
2016 – 2018

기술

TypeScript, JavaScript
React.js, Next.js, React Native (Expo)
TanStack Query, Zustand, Zod
Vitest, Vite
Claude Code

자격증

정보처리기사 · SQLD · ADSP
네트워크 관리자 2급
투자자산운용사

SafeSearch

CCTV 영상에서 AI로 특정 인물을 찾고, 그 사람이
거쳐 간 카메라를 지도에 표시하는 웹 서비스

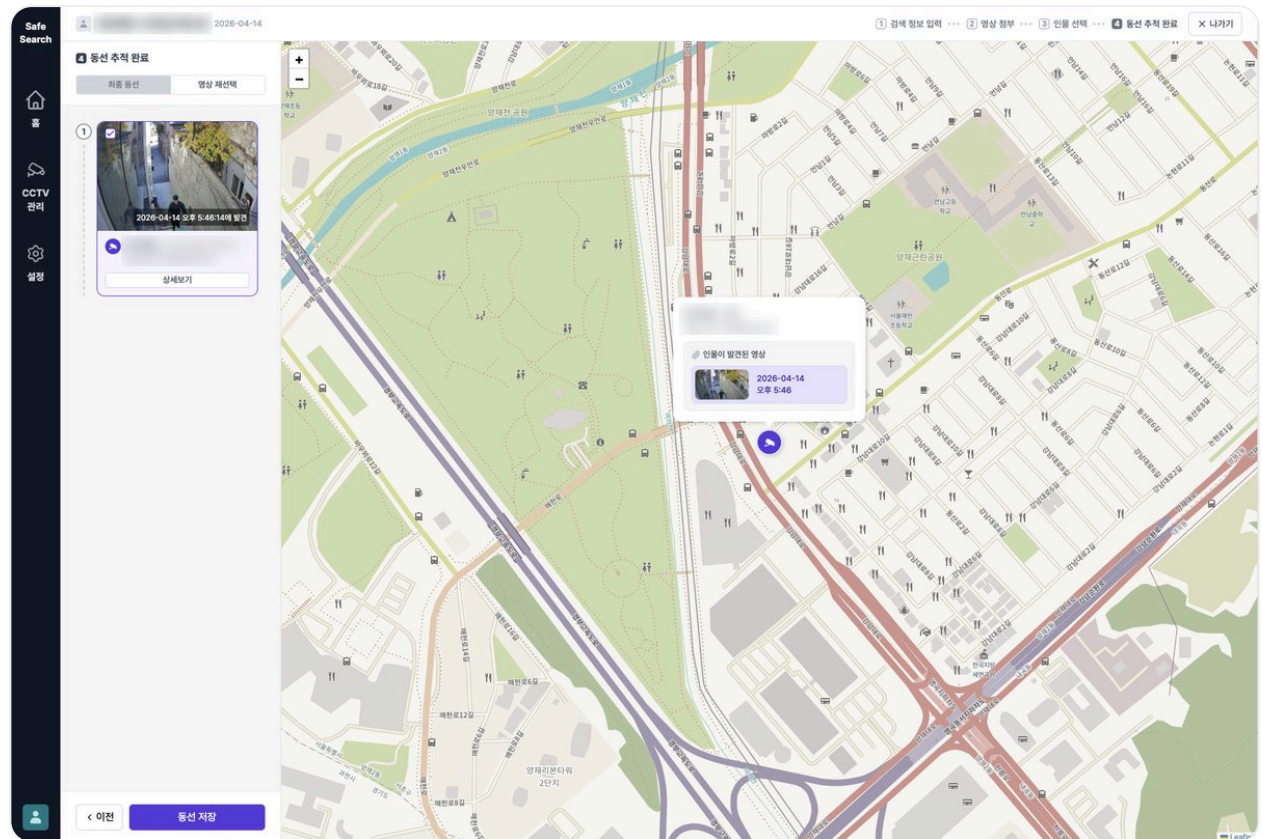
배경

사람 한 명을 찾으려 해도 여러 카메라의 영상을 눈으로 하나씩 돌려
봐야 합니다. SafeSearch는 이 일을 AI 분석으로 대신합니다. 보안
관제나 사건 분석을 맡은 담당자가 찾을 사람의 사진과 CCTV
영상을 올려 씁니다.

저는 프론트엔드 전체를 맡아 지도, 영상 업로드, 분석 진행 화면,
서버와 주고받는 코드를 만들었습니다. 영상이 크고 분석에도
시간이 걸려서, 올리고와 분석 대기, 취소와 재시작이 서로 엉키기
쉬웠습니다.

스택 React, TypeScript, Vite, TanStack Query, Leaflet

회사 SAFEAI



01 SafeSearch · 화면과 기능

화면 왼쪽은 목록, 오른쪽은 지도입니다. 사람 찾기 한 건을 프로젝트 하나로 만들고, 검색 정보 입력부터 동선 추적까지 네 단계를 차례로 거칩니다.

CCTV 등록

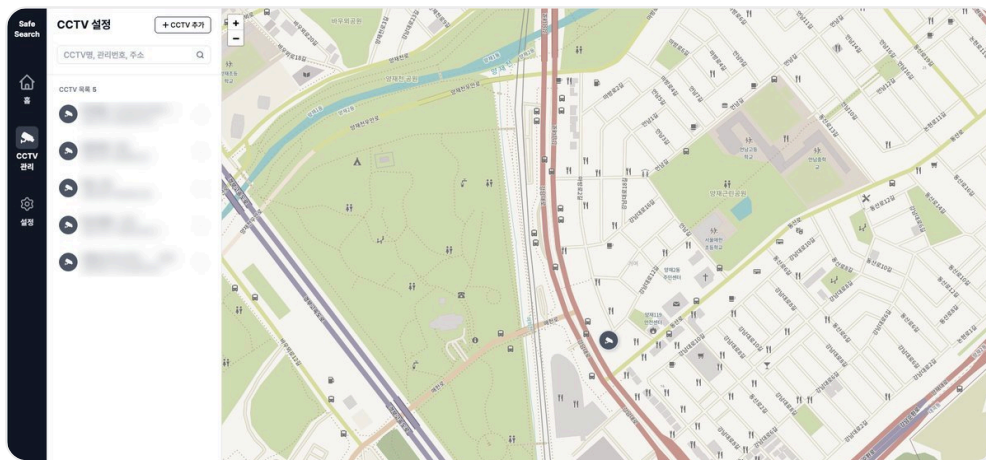
카메라 이름과 주소를 등록하면 지도에 마커로 나타납니다.
이름, 관리번호, 주소로 검색해 찾습니다.

영상 첨부

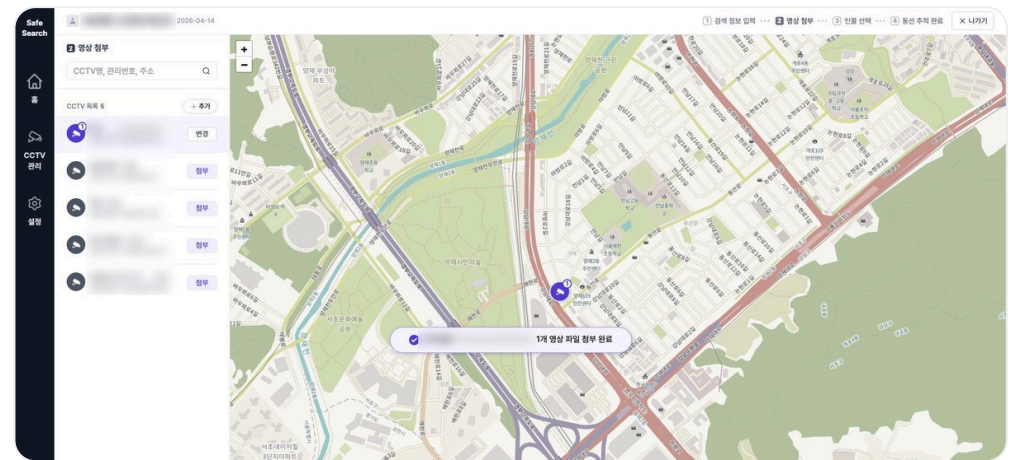
지도를 옮기고 "현 지도에서 검색"을 누르면 보이는 범위의
카메라만 나옵니다. 영상은 올리다 끊어도 멈춘 곳부터 이어
올립니다.

인물 선택과 동선

AI가 찾아 준 비슷한 사람 중 한 명을 고릅니다. 그 사람이 찍힌
장면을 시간순으로 이어 지도에 이동 경로를 그립니다.



CCTV 설정: 등록된 카메라 목록과 지도



영상 첨부: 지도에서 고른 카메라에 영상을 붙인 화면

취소하고 다시 시작해도 화면이 꼬이지 않게

나가기로 분석을 멈추고 곧바로 다시 시작하면, 앞 작업의 응답이 늦게 도착해 새 화면을 덮어쓸 수 있습니다.

01 단계를 한곳에 모음

단계와 다음 단계를 표 하나에 정해 둔 구조(상태머신)에
올리기, 분석 대기, 취소, 재시작을 모두 모았습니다. 순서가
틀린 요청을 막는 검사까지는 넣지 않았습니다.

02 실행마다 번호

흐름을 시작할 때마다 번호를 붙여 요청에 함께 싣고, 번호가
지금 실행과 다르면 버립니다. 취소한 작업의 응답이 늦게
와도 화면은 마지막 실행만 따라갑니다.

03 화면 없이 테스트

이 구조는 React 없이 도는 함수로 떼어 내, 화면을 띄우지
않고 입력과 결과만으로 테스트합니다.

SafeVision

매장 카메라 영상에서 AI가 상품이 빠진 매대를 찾아 알려 주는 서비스의 관리자 웹과 앱

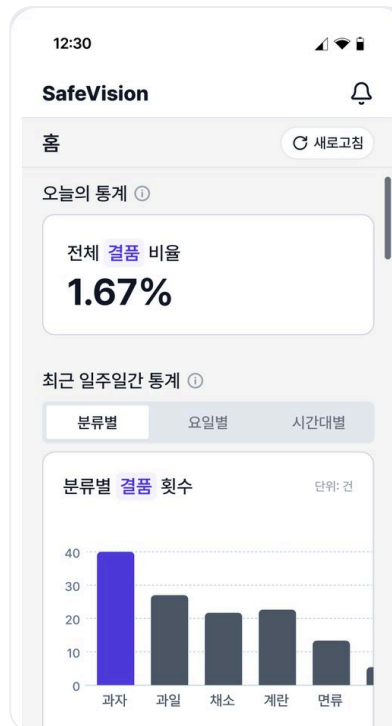
배경

매대에서 상품이 떨어져 비어 있는 상태를 결품이라고 합니다. AI가 카메라 영상에서 결품을 찾으면 관리자는 웹 대시보드와 휴대폰 알림으로 확인합니다. 대형마트 L사 매장에서 실제 사용자를 대상으로 현장 실증(PoC)을 운영했고, iOS와 Android 스토어에 출시했습니다.

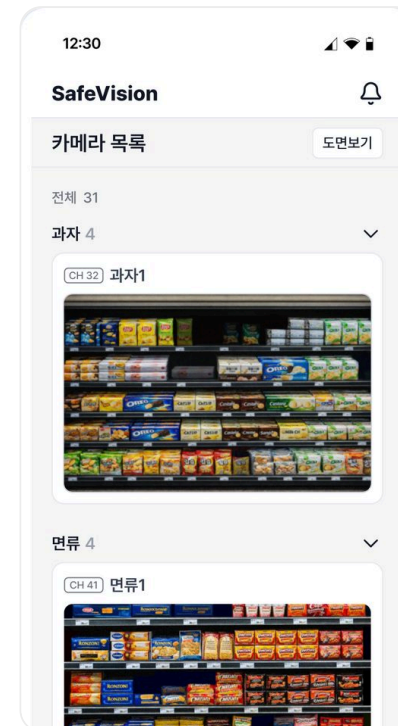
저는 관리자 웹과 앱의 프론트엔드를 모두 맡았습니다. 앱은 새로 만들지 않고 웹 대시보드를 앱 화면 안에 그대로 띄웠고, 알림과 화면 전환처럼 웹으로 안 되는 부분만 앱 코드로 만들었습니다.

스택 React, TypeScript, Vite, TanStack Query, Zustand, React Native, Expo

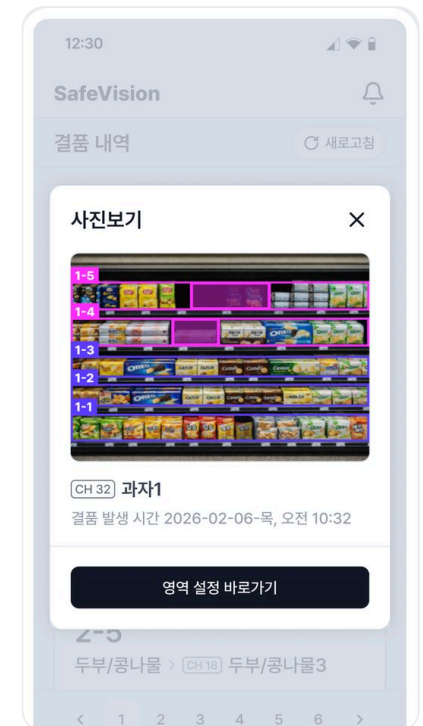
회사 SAFEAI



홈: 오늘의 결품 비율과 분류별 결품 횟수



카메라 목록: 매대 분류별로 묶은 카메라



결품 내역: 감지 영역을 표시한 매대 사진

02 SafeVision · 화면과 기능

홈에서 오늘의 결품 현황을 보고, 카메라 목록과 결품 내역에서 매대 사진을 확인합니다. 관리자 웹 전체는 20개 화면입니다.

결품 통계

전체 결품 비율과 분류별 · 요일별 · 시간대별 결품 횟수를 그래프로 봅니다. 새로 감지된 결과는 새로고침하지 않아도 화면에 들어옵니다.

결품 알림

결품이 감지되면 휴대폰으로 알림이 옵니다. 인터넷이 끊겨도 알림 목록에서 최근 알림을 볼 수 있습니다.

카메라와 결품 사진

카메라를 매대 분류별로 묶어 보여 줍니다. 결품이 나면 그 시각과 함께 감지 영역이 표시된 매대 사진을 봅니다.

앱과 맞물리는 화면 전환

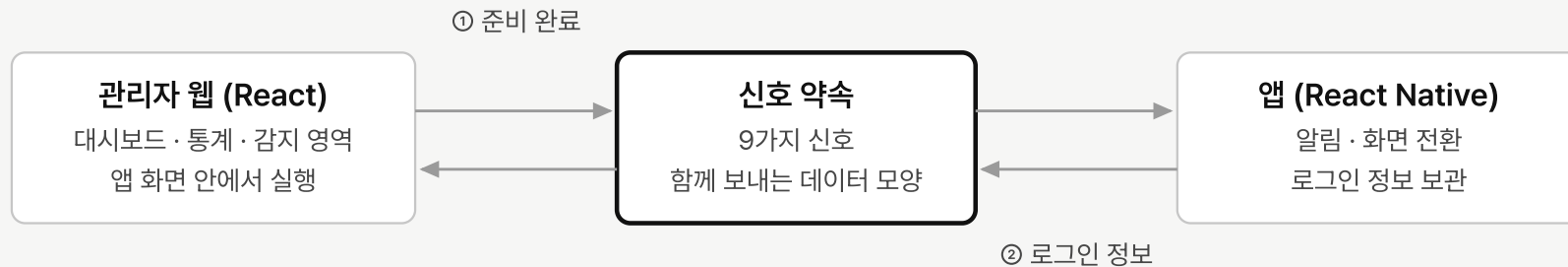
웹 화면이 상세 페이지로 넘어가면 앱 아래쪽 탭을 숨기고 상세 화면용 머릿글을 띄웁니다. 안드로이드 뒤로 가기 버튼도 웹 화면 흐름을 따라갑니다.

감지 영역 지정

카메라 화면 위에 AI가 살퍼볼 칸을 끌어서 그리고 크기를 바꿉니다. 화면 크기가 달라도 칸이 어긋나지 않게 원본 사진 기준 좌표로 저장합니다.

노코드 관리자 웹을 React로 다시 만들고 앱과 잇기

관리자 웹은 원래 코드 없이 화면을 만드는 노코드 도구(WeWeb)로 만들어져 있었습니다. 도구의 실행 코드와 편집기 코드까지 통째로 내려받아 무거웠고, 코드가 어떤 순서로 도는지 볼 수 없어 앱 안에서 생긴 문제를 찾기 어려웠습니다. 웹 전체를 React로 다시 만든 뒤, 같은 대시보드 기준으로 첫 화면이 뜨는 시간(LCP)이 19.27초에서 3.29초로 줄었습니다.



웹과 앱은 약속에 적힌 9가지 신호로만 주고받습니다. 없는 신호나 모양이 틀린 데이터는 실행하기 전에 오류로 드러나고, 앱은 웹이 준비 완료를 알린 뒤에 로그인 정보를 보냅니다.

dong-skills

업무에서 쓰는 Claude Code 스킬 8개를 모아
플러그인으로 배포하는 스킬 모음

만든 이유

AI 에이전트(Claude Code)는 새 대화를 열 때마다 우리 팀이
일하는 방식을 모르는 상태에서 시작합니다. "문서는 누가
읽는지부터 정하고 쓴다", "작업 공유는 비개발자도 읽게 쓴다" 같은
요구를 대화마다 다시 설명하는 대신, 한 번 문서로 적어 두고 AI가
알아서 꺼내 읽게 만들었습니다.

이 문서 한 장을 스킬이라고 부릅니다. 업무에서 쓰던 8개를 묶어
명령 한 줄로 설치할 수 있게 공개했습니다.

형태 Claude Code 플러그인 (Markdown 문서, Python 검사
스크립트)

GitHub github.com/easyDong19/dong-skills

문서 작성	
choosing-doc-type	쓰기 전에 누가 무엇을 하려고 읽는 글인지 정해 문서 종류를 하나로 고릅니다
doc-structure-review	초안의 목차를 다섯 가지 원칙으로 점검하고, 고친 목차 하나를 내놓습니다
doc-sentence-polish	뜻은 그대로 두고 읽기 어려운 문장만 골라 쉬운 표현으로 고칩니다
explaining-with-domain-context	코드를 설명할 때 파일 목록이 아니라 풀려던 문제부터 원인과 결과 순서로 설명합니다
작업 공유	
daily-scrum	그날의 코드 작업 기록을 기획·디자인 직군도 읽는 "어느 화면의 무엇이 바뀌었나" 메모로 바꿉니다
qa-report	화면 캡처와 점검 항목을 모아, 사람이 항목마다 승인·반려를 고르는 검수 리포트로 만듭니다
학습 자료	
study-explainer	공부한 주제를 처음 보는 사람도 따라오도록 배경·비유·그림을 채운 학습 페이지로 만듭니다
a4-study-sheet	공부한 내용을 시험·면접 직전에 볼 흑백 A4 한 장 요약으로 만듭니다

03 dong-skills · 쓰는 방식

스킬을 설치해 두면 따로 부르지 않아도 됩니다. 요청 내용이 스킬 설명과 맞으면 AI가 알아서 골라 읽고 그 순서대로 일합니다.

문서 일은 세 단계로 나눴습니다

문서 종류 정하기, 목차 점검, 문장 다듬기를 스킬 셋이 차례로 맡습니다. 한 스킬이 다른 스킬의 일을 넘보지 않게 나눴습니다.

개발자 말을 쉬운 말로 바꿉니다

스크럼 메모 스킬에는 바꿔 쓸 말의 예가 들어 있습니다. "캐시 폐기 일원화"는 "로그아웃하면 앞 계정 데이터가 남아 보이던 문제 해결"로 씁니다.

모르는 건 추측하지 않고 묻습니다

어느 화면인지 알 수 없는 작업은 지어내지 않고 사람에게 먼저 묻게 했습니다.



QA 리포트 스킬이 만든 검수 리포트 예시(샘플 데이터). 오른쪽 위 "판정 대기" 칸은 사람만 채웁니다

AI가 쓴 결과를 사람에게 넘기기 전에

QA 리포트 스킬에서는 AI가 하는 일과 사람이 하는 일을 파일로 나눴습니다. 검사를 통과해야 리포트가 만들어지고, 승인과 반려는 사람만 저장합니다.



dongmodoro

주간·오늘 계획과 뽀모도로 타이머를 한 화면에 둔
macOS 데스크톱 앱

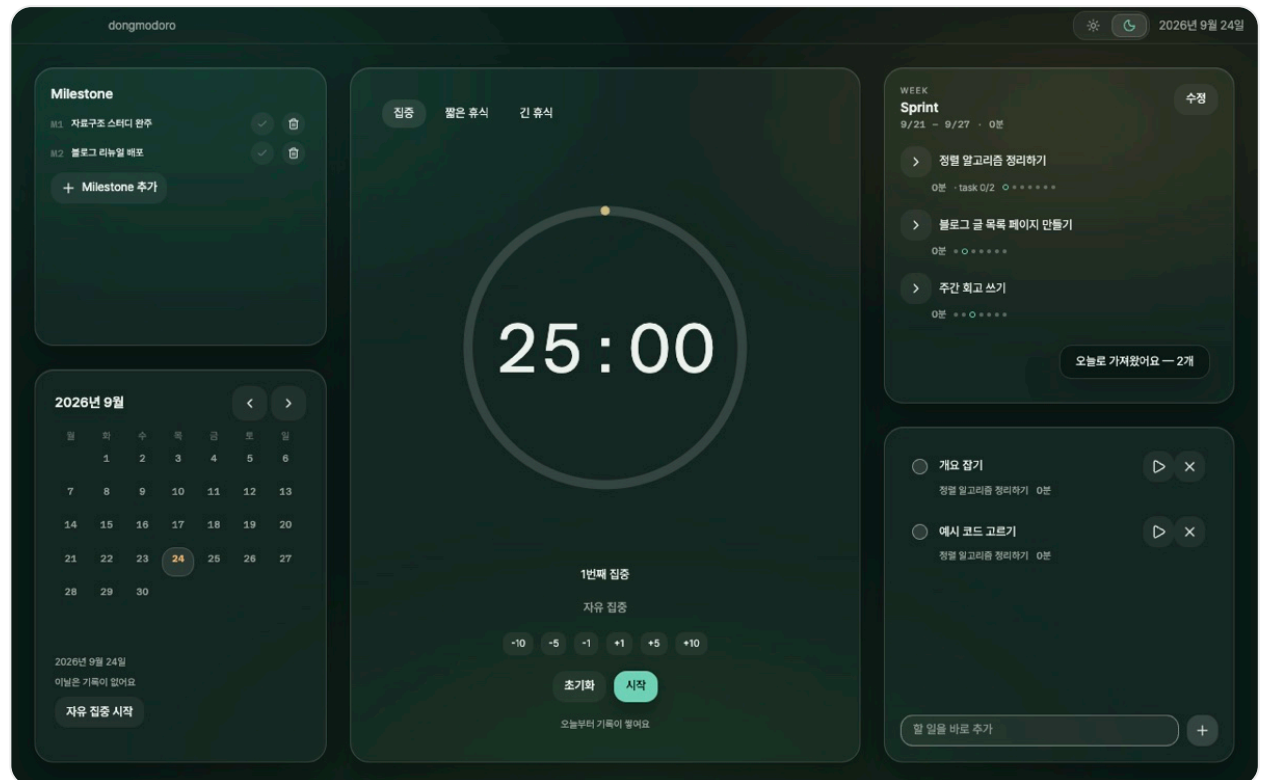
만든 이유

AI에게 일을 시키면서 창을 여러 개 띄워 두고 작업을 동시에
굴렸더니 집중이 자주 끊겼습니다. 게다가 계획은 노선에, 타이머는
다른 앱에 있어서 하루를 시작할 때마다 둘 사이를 오가야 했습니다.

그래서 계획과 실행이 한 화면에서 이어지는 앱을 직접
만들었습니다. 로그인도 동기화도 없이 내 컴퓨터에서만
돌아갑니다.

스택 Electron, React 19, TypeScript, SQLite, Vitest,
Playwright, Claude Code

GitHub github.com/easyDong19/dongmodoro



04 dongmodoro · 화면과 기능

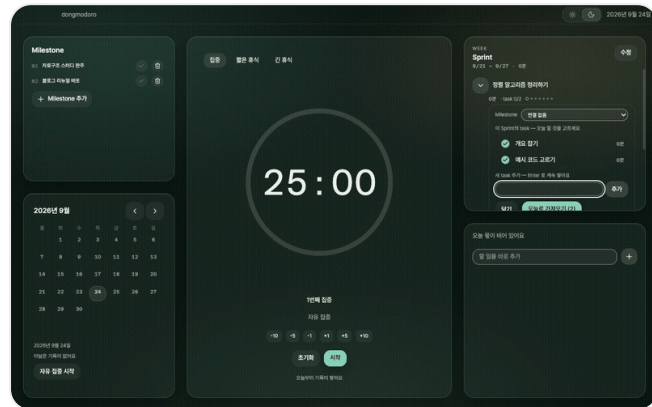
한 화면을 세 칸으로 나눴습니다. 왼쪽은 이번 달, 가운데는 지금 돌리는 타이머, 오른쪽은 이번 주와 오늘 할 일입니다.

주간 계획

이번 주에 할 일을 제목으로 적고 요일에 놓습니다. 계획할 때 시간을 미리 적는 칸은 두지 않았습니다.

주간 정산

주가 끝나면 남은 일을 한 화면에서 다음 주로 넘기거나 지웁니다. 몇 주를 비워도 화면은 하나입니다.



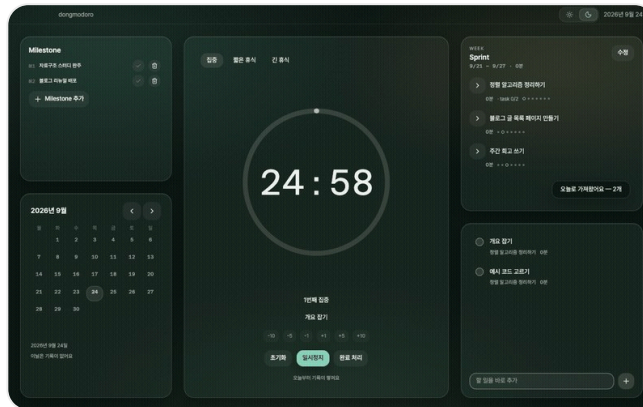
주간 계획에서 오늘 할 일을 가져오는 화면

오늘 목록

주간 계획에서 오늘 할 일을 쏘개 가져옵니다. 급한 일은 바로 추가할 수 있습니다.

마일스톤과 달력

한 달 동안 끝낼 일을 적고, 기록이 있는 날은 달력에 점으로 찍힙니다. 날짜를 누르면 그날의 계획이 다시 보입니다.



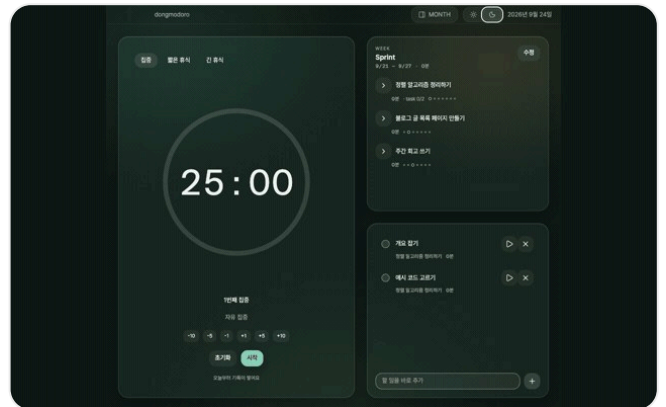
고른 일로 타이머가 도는 화면

뽀모도로 타이머

오늘 목록에서 하나를 골라 바로 돌립니다. 실제로 집중한 시간이 주간 계획 카드에 쌓입니다.

좁은 창과 다크 모드

창을 화면 절반으로 줄이면 달력이 접히고 타이머가 남습니다. 다크와 라이트 테마를 고를 수 있습니다.



창을 좁히면 타이머와 이번 주 목록만 남음

AI에게 코드를 맡기며 씩운 검사 장치

커밋 106개 중 90개를 AI 에이전트(Claude Code)와 함께 썼습니다. AI가 정해 둔 규칙을 어기지 않도록 검사를 세 겹으로 씩웠습니다.

01

명령을 실행하기 직전

완성본에 바로 올리기, 기록 강제로 덮어쓰기 같은 명령은 막습니다. 배포처럼 중요한 일은 실행 전에 사람에게 묻고, 기획 원본 문서는 AI가 고칠 수 없게 잠가 두었습니다.

02

코드를 저장할 때

저장할 때마다 코드 형식과 기록 메시지 규칙을 자동으로 검사합니다.

03

GitHub에 올린 뒤

테스트와 빌드를 자동으로 돌립니다. 배포 파일은 초안으로만 만들어지고, 공개 버튼은 사람이 누릅니다.

Scratchpad

코딩테스트 C++ 템플릿을 외워 치는 연습에 맞춰
직접 만든 macOS 에디터

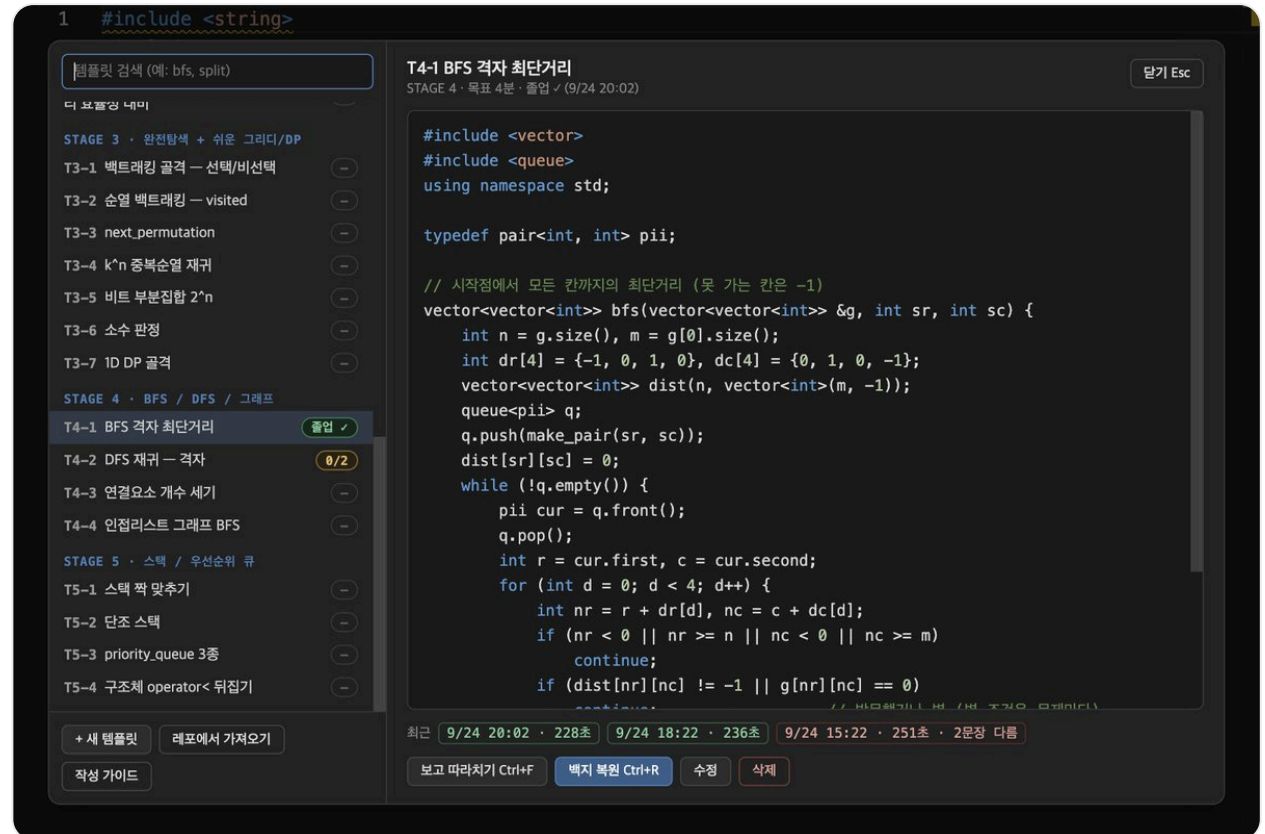
만든 이유

코딩테스트를 C++로 준비하면서, 자주 나오는 코드 뼈대(BFS, 백트래킹 등)를 템플릿으로 정리해 손에 익을 때까지 외워 쳤습니다.

코딩테스트 사이트에는 자동완성이 없어서 실전과 같은 조건으로 연습하고 싶었습니다. 그래서 보고 치고, 가리고 치고, 채점까지 한 곳에서 되는 에디터를 제 공부 방식에 맞춰 직접 만들었습니다.

스택 Electron, Monaco Editor, xterm.js, node-pty, clangd, Claude Code

GitHub github.com/easyDong19/scratchpad

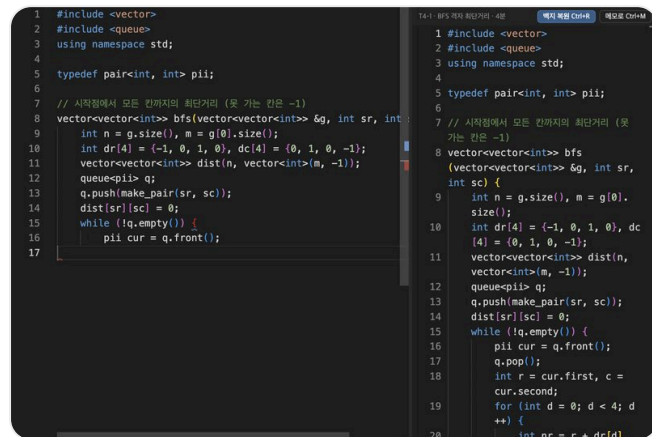


05 Scratchpad · 화면과 기능

템플릿 하나를 세 번에 걸쳐 익힙니다. 보고 따라 치고, 가리고 쳐음부터 다시 치고, 제출하면 채점합니다. 목표 시간 안에 두 번 연속 맞으면 졸업입니다.

바로 실행

코드를 쓰다가 단축키 하나로 컴파일하고 실행합니다. 입력값도 아래 터미널에 바로 칩니다.

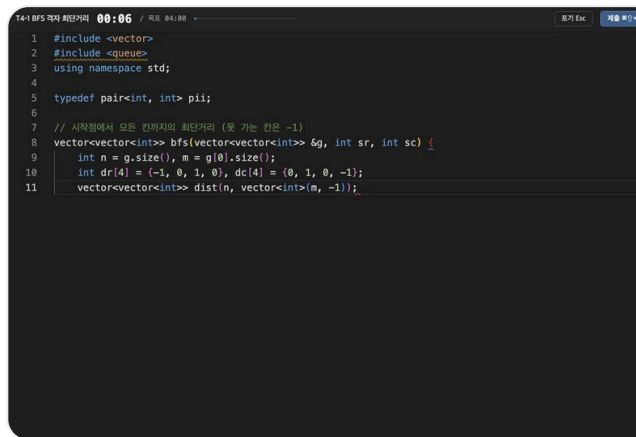


The image shows the Scratchpad interface with a code editor on the left and a terminal on the right. The code is a C++ program for a BFS algorithm. The terminal shows the output of the program.

보고 따라 치기: 오른쪽 템플릿을 보며 왼쪽에 칩

실전 모드

코딩테스트 사이트처럼 자동완성을 끌 수 있고, 가리고 칠 때는 자동으로 꺼집니다.

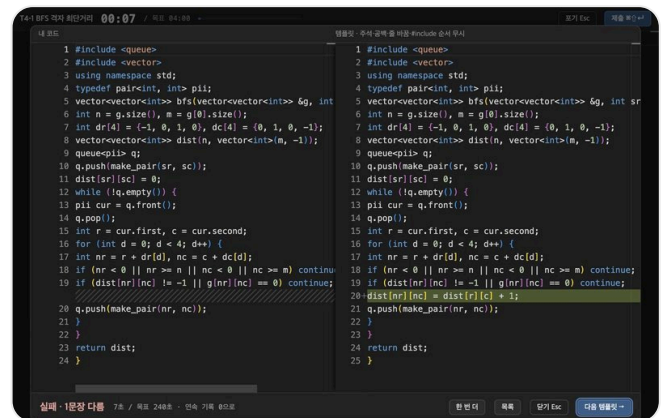


The image shows the Scratchpad interface with a timer at the top and a 'Focus' button. The code is the same as in the previous image.

가리고 치기: 위쪽에 목표 시간과 경과 시간

내용으로 채점

줄 바꿈이나 공백, 주석이 달라도 내용이 같으면 맞다고 봅니다. 틀린 곳은 한 문장 단위로 짚어 줍니다.



The image shows the Scratchpad interface with a code editor on the left and a terminal on the right. The code is the same as in the previous images. The terminal shows the output of the program.

채점: 빠뜨린 한 문장을 노란 줄로 표시

연습할 템플릿은 AI에게 맡겼습니다

템플릿을 손으로 다 쓰기엔 양이 많아 AI(Claude Code)에게 맡겼습니다. 다만 형식이 조금만 틀려도 앱이 조용히 빼먹어서, 아래처럼 준비했습니다.

01

작성 설명서

템플릿 형식과 목표 시간 기준을 스킬 문서로 적었습니다. 이 문서만 준 다른 AI에게 써 보게 하고, 헛갈려한 곳을 고쳤습니다.

02

앱과 같은 검사기

앱이 파일을 읽는 코드를 그대로 쓰는 검사기를 만들어, AI가 경고가 0개가 될 때까지 고치게 했습니다.

03

스킬 없이 쓸 때

Claude Code가 없는 사람도 쓸 수 있게, 앱 안에서 AI에게 붙여넣을 요청문을 복사 버튼으로 줍니다.